Laboratory Exercise 01

The Mechanics of C

What You Are Doing Today

This exercise has four parts. You will repair a program that does not build, write a second program from nothing, induce a memory leak deliberately and then correct it, and finally diagnose two programs that compile and run but behave incorrectly.

Nothing in this exercise is new material. Every concept required was covered in `MODULE 01`. If you encounter difficulties, you are encouraged to review your notes and discuss the relevant concepts with your classmates before seeking further assistance.

Part	What it is
A	Make a provided program build and run.
B	Write a program from nothing, then make it leak.
C	Find the defect in two broken programs
D	Write, by hand, what you learned

Before You Begin

1. Create a working folder for this session and work inside it.

   ```
   C:\Users\<YourUserName>\datastruct\labex01
   ```

   Do not work on the `Desktop` or in the root of `C` folder, and do not use a folder whose name contains spaces.

2. Copy the provided folder into it. You will be told where to obtain it. It contains `partA\` and `partC\`.

3. Open a terminal *in that folder*.

4. Confirm that your compiler is installed before you need it:

   ```
   gcc --version
   ```

   If the compiler is not installed, notify me. You will install it under my guidance.

5. **On a short bondpaper, write your heading.** Name, student number, section and date, in the top left corner, in blue or black ink. Keep the sheet beside your keyboard for the whole session. The **front** is a log that you add to during Parts A and C. The **back** is the essay you write at the end.

Use these flags for all the builds you will do for this laboratory exercise

```
gcc -Wall -Wextra -std=c17 -g files -o name.exe
```

PREPARED BY:

nlaldllllolibv

ENGR. VIOLDAN E. BAYOCOT
Instructor
University of San Jose - Recoletos

Rules

- **Everything happens in this room.** No work is taken home and no work is completed later.
- **No internet.** Your notes, the lessons, and your own understanding. That is the complete list.
- **Discussion is permitted.** Explaining a pointer to a classmate is the fastest way to discover whether you understand it. Reading a classmate's screen and reproducing what you saw is not, and the difference is apparent from behind you.
- **Work is credited only after I have checked it**. See below.

Verification and Marking

When you complete a part, notify me. I will ask you to explain some aspect of your work, usually one question and occasionally two. The purpose is not to catch you out. It is to establish whether the code on your screen follows from your own understanding.

You may be asked to explain any line you have written, including lines taken from the class discussion.

If you cannot answer, you do not lose the mark at that point. You will instead revisit the concept and work through it, because that is the purpose of this session.

Part A. Make It Build

Inside `partA\` you have five files: `hello.c`, `stats.h`, `stats.c`, `main.c`, and `input.txt`.

A0. Prove the toolchain

Compile and run `hello.c`. This file is correct. It is provided so that you can confirm the compiler works before investigating code that does not.

A1. Make the program build

`main.c`, `stats.c` and `stats.h` together form one program that computes statistics over a list of numbers. It does not build. Several defects are distributed across `stats.h` and `stats.c`, and one function that `main.c` calls has not been written. `main.c` is correct and must not be edited. Treat it as the specification: whatever `main.c` requires is what the remaining files must provide. The arithmetic in `stats.c` is correct. Do not rewrite it. Every defect is a small one.

Correct them one at a time, recompiling after each. The list of errors reduces substantially each time the first one is resolved.

- Some defects are indicated almost directly by the compiler, provided you read the entire message, including the note beneath it.
- One is a disagreement between two files. Each file appears reasonable on its own.
- One does not appear until every other error has been cleared and the compiler has fallen silent. It originates from a different program in the toolchain.
- One function has not been written at all. Write it. It requires two lines: `main.c` gives its name, and `stats.h` gives its shape.

On your log sheet, before proceeding: record the first error message you saw, copied exactly, together with the change that cleared it. Copy it while it is still on your screen, and scroll back if you have already cleared it.

A2. Run it, and feed it a file

Once it builds with no compiler output at all, run it three ways:

```
cmd /c ".\stats.exe < input.txt"
cmd /c ".\stats.exe < input.txt > output.txt"
.\stats.exe
```

The expected result for `input.txt` is stated at the top of `main.c`. Compare yours against it.

The second command directs the results into a file. Open `output.txt` and compare it with what appeared on your screen. Identify it, and be prepared to explain why.

The third command has no file to read. Determine what it is waiting for, and how to terminate it.

A3. Look inside it

Set a breakpoint on `main`, run to it, advance a few lines, and print the value of `n` after `readValues` returns.

```
gdb .\stats.exe
(gdb) break main
(gdb) run
(gdb) next
(gdb) print n
(gdb) quit
```

You will need this in `Part C`, and you will rely on it far more heavily once the data structures grow.

If `gdb` reports that it has no line number information, your compile command omitted something.

Part B. Build It Yourself

Write a new file, `lab01-yourlastname.c`, from nothing. No starter file is provided for this part. You are given what the program must *do*. How it does so is your responsibility, which is the purpose of the exercise.

Your program prints five labelled blocks, in the order below. Complete and have each one checked before beginning the next.

B1. A pointer is an address

Print a variable's value, its address, and its value read back through a pointer.

Then call a function that receives the *address* of that variable and sets it to zero, and print the variable again to demonstrate that it changed. The function returns `void` and does not use a global variable.

- If the function's parameter is an `int`, the variable in `main` will not change, and you already know the reason.
- Do not use `scanf` in this block. Assign the value directly.

B2. An array decays

Declare an array of at least six `int`s in `main`. Print the number of elements it holds, computed rather than written in as a literal.

Then pass it to a function that prints the whole array correctly. Inside that function, also print what `sizeof` reports for the parameter.

Two values must appear on screen together: the size obtained in `main` and the size obtained inside the function. They will differ. In a comment above the function, explain why in your own words.

- The function must be told how many elements it has received. Determine why it cannot establish this for itself, and the required parameter list follows.
- Somewhere in this block, print one element in two ways: using brackets, and without using brackets at all.

B3. The heap

Ask the user for a count. Allocate exactly that many `int`s on the heap, fill them with values that are computed rather than written out, print them, and release the memory.

- `malloc` counts *bytes*. It does not count `int`s. This is the most common defect in this exercise.
- `malloc` can fail. Check its result every time, and respond appropriately when it does.
- After releasing the memory, one further line makes the pointer honest about it. Write that line.

B4. Ownership

Move the allocation out of `main` and into a function that allocates the block, fills it, and returns it. `main` then prints the block and releases it.

In a comment on that function, state in one sentence **who is responsible for releasing what it returns**. You will be asked about this.

B5. Demonstrating a memory leak

This block is demonstrated, not submitted. Do not begin it until `B1` to `B4` have been checked.

Write a *separate* file, `leak.c`. It runs a loop. On each iteration it requests a block of approximately one megabyte and does not release it. It reports its progress so that the rate is visible, and it terminates on its own after a few hundred iterations rather than running until the machine is exhausted.

1. Open `Task Manager`, select the `Details` tab, and add the `Memory` column. Position it beside your terminal so that both are visible at once.
2. Run your program and observe the memory figure rise. **Notify me while it is still rising.**
3. You will be asked one question: *which line makes this a leak, rather than a program that simply uses a large amount of memory?* The answer is not the missing `free`. Consider what happens to the address on each iteration.
4. Then add the missing line, rebuild, and run it again with Task Manager still open. The memory figure remains flat. Demonstrate this as well.

> **Two practical constraints**
>
> Stop the program with `Ctrl+C` as soon as it has been observed. Do not leave it running while you proceed to Part C, and do not raise the limit to see what happens.
>
> The program must write to the memory it requests, not merely request it. `Memory` that is never written to may never be committed by the `operating system`, in which case `Task Manager` shows nothing and the demonstration fails.

Part C. Two Broken Programs

Work in `partC\`. It contains twelve files, `bug00.c` through `bug11.c`.

Every one of them compiles. Every one of them runs. Every one of them is incorrect. Each file's comment header states what the program was intended to do and what it actually does. It does not state why, which is your task.

C1. Your assigned file

Take the **last two digits** of your student number and divide by 12. The remainder identifies your file. This is the same rule used in `Workshop 01`.

Remainder	Your file	Remainder	Your file
0	`bug00.c`	6	`bug06.c`
1	`bug01.c`	7	`bug07.c`
2	`bug02.c`	8	`bug08.c`
3	`bug03.c`	9	`bug09.c`
4	`bug04.c`	10	`bug10.c`
5	`bug05.c`	11	`bug11.c`

C2. One file of your own choosing

Select any *other* file from the twelve and repeat the procedure. Choose it before examining any of them, and do not select the file your seatmate has chosen.

The rules for both

1. **Each file contains exactly one defect.** If your correction affects four lines, you have not yet identified it.
2. **Do not rewrite the program.** A correct program written from scratch receives no credit here, because it does not demonstrate that anything was found. The smallest change that makes the program correct is the answer.
3. **Run it before reading it.** Observe the failure first, then investigate.
4. **Compile with the full set of flags.** Four of the twelve produce compiler warnings, and reading the warning is a legitimate method of locating the defect. Ignoring it is not.
5. Some terminate without printing anything. `gdb` and `backtrace` will identify where. This is the purpose of A3.

The line on which a program fails is frequently not the line that is wrong

Several of these twelve fail at a considerable distance from the actual mistake. The program appears to work, completes its output, and then terminates abnormally on the way out.

When this occurs, the question is not what is wrong with this line, but what an earlier line did that made this line impossible. This is the most useful diagnostic habit in the session, and it is the reason Part C exists.

On your log sheet, one entry per file, written before you start the next one:

- The file name, and the symptom in your own words rather than copied from the header
- The line that is wrong, given as a line number
- Why it is wrong, in one sentence
- The change you made

Part D. Write It Down, By Hand

The front of your sheet has been accumulating entries throughout the session. This part is the back, and it is the final task. Do not begin it until ***`Parts A, B and C`*** *have been checked.*

Side	What is on it
Front: your log	Already written, as you went: the heading block from the start of the session, the `Part A` error message and its fix, and your two `Part C` entries. Confirm it is complete before you turn over
Back: the essay	The three questions below, answered in your own words.

The three questions

1. Of the two programs you repaired in `Part C`. Describe **how you found the defect**, including the steps that led nowhere. Do not describe what the defect turned out to be.
2. The defects in `Part A` were all caught by the compiler. The defects in `Part C` were not, and several of those programs ran to completion and exited normally. Explain the difference between these two categories of defect, and state which you expect to cause more difficulty when you build `ADTs` later in the course.
3. Your program in `B5` printed everything it was intended to print and exited with status `0`. By any test applied to its output, it worked. Explain, in your own words, why it was nonetheless incorrect.

> **What "in your own words" means here**
>
> It means that the reader can tell you were thinking. An answer that returns the phrasing of the lesson notes is not wrong, but neither is it evidence of understanding, and it will be handed back. A short, honest answer is preferable to a long, impressive one.

Submit the sheet to me. I will review it. If it is accepted, `Laboratory Exercise 01` is complete and you may proceed to the next assessment. If it is not, I will identify the question that requires more, and you will rewrite that portion. There is no penalty for this, but the exercise is not complete until the paper is accepted.

Check This Before Requesting Verification

Work through this list yourself first. It covers most of what I will examine, and it will prevent you from being sent back for something that takes twenty seconds to correct.

Every part

- ☐ Compiled with `-Wall -Wextra -std=c17 -g`, and the compiler produced *no output*
- ☐ The program ran in its current form, not five edits ago
- ☐ I understand and can explain every line on the screen

Part A

- ☐ `stats.exe` prints the same numbers as the block at the top of `main.c`
- ☐ I did not edit `main.c`
- ☐ I know which line went to the file and which line went to the screen, and why
- ☐ I have the first error message written on my log sheet, copied exactly

Part B

- ☐ All five blocks print, labelled, in order
- ☐ Two different `sizeof` results appear on screen together, with a comment explaining why
- ☐ Every `malloc` is checked against `NULL` before it is used
- ☐ Every `malloc` has a matching `free`, and every `free` is followed by `= NULL`
- ☐ No array is passed to a function without its length beside it
- ☐ The comment in B4 states who releases the returned block
- ☐ `leak.c` has been demonstrated *leaking*, and then demonstrated again *corrected*
- ☐ `leak.c` is no longer running

Part C

- ☐ I repaired the file my student number assigned me, and one other
- ☐ Each correction is the smallest change that works, and I did not rewrite either program
- ☐ Both repaired programs compile silently and produce the output their header describes
- ☐ My log sheet has a line number and a one-sentence reason for each

Part D

- ☐ Front and back both completed, by hand, in blue or black ink
- ☐ Name, student number, section and date at the top left of the front
- ☐ All three questions answered, in my own words

How This Is Marked

This exercise is worth 50 points. It is checked at your machine while you work, and the mark is recorded before you leave the room.

What is being checked	Points
Part A: the program builds silently and prints the correct results	8
Part A: the missing function written correctly	2
Part A: redirection understood, including which line went where and why	2
B1: value, address, and value through a pointer, with the caller's variable changed by a function	4
B2: both `sizeof` results shown, length passed as a parameter, comment explains the difference	5
B3: correct byte count, `NULL` checked, memory released, pointer cleared	6
B4: allocation returned from a function, ownership stated in a comment	4
B5: leak demonstrated live, question answered, then corrected and demonstrated again	6
C1: assigned file, defect found, minimal correction, reason correct	6
C2: chosen file, defect found, minimal correction, reason correct	5
Part D: log sheet complete and legible	2
Total	**50**

The three questions on the back of your sheet carry no points. They determine whether you have finished. An accepted paper means Laboratory Exercise 01 is complete and you may proceed to the next assessment.

The concepts covered in this exercise are important foundations for the work that follows, so take your time, review the relevant material, and seek clarification when needed..

PREPARED BY:

nlaldlllolibv

ENGR. VIOLDAN E. BAYOCOT
Instructor
University of San Jose - Recoletos